

BÀI TẬP 3

3.1

- Kết quả in ra màn hình là : Dog barks
- Vì phương thức MakeSound() được khai báo là virtual ở lớp cha và được override ở lớp con, C# sẽ áp dụng cơ chế liên kết muộn. Lúc chạy, chương trình dựa vào kiểu đối tượng thực tế (Dog) để gọi phương thức tương ứng, thay vì kiểu tham chiếu (Animal).

3.2

- Kết quả in ra màn hình là: Animal makes a sound
- override (Ghi đè): Phá vỡ hoàn toàn mối liên kết với phương thức lớp cha. Dù dùng biến kiểu Animal để gọi, nó vẫn tìm đến hàm đã ghi đè của Dog.
- new (Ẩn hàm): Chỉ tạo ra một phương thức mới trùng tên ở lớp Dog chứ không liên quan gì đến hàm của Animal. Do đó, nếu gọi thông qua biến kiểu lớp cha (Animal myPet), chương trình sẽ chạy hàm của lớp cha Animal. Nó chỉ chạy ra Dog barks khi bạn ép kiểu biến hoặc dùng biến kiểu cụ thể là Dog (Dog myDog = new Dog();).

3.3

- Điều gì xảy ra: Khi bạn khởi tạo lớp con Dog, Constructor của lớp cha Animal sẽ chạy trước. Nếu lớp cha gọi một phương thức virtual, trình biên dịch C# sẽ lập tức nhảy thẳng xuống thực hiện phương thức đã override trong lớp con Dog ngay tại thời điểm đó, dù lúc này Constructor của lớp con Dog còn chưa được chạy và các thuộc tính của Dog chưa kịp khởi tạo.
- Nó phá vỡ tính toàn vẹn và an toàn dữ liệu của đối tượng
- Nếu phương thức MakeSound() ở lớp con truy cập vào bất kỳ trường dữ liệu hoặc thuộc tính nào của Dog, các giá trị đó lúc này đều đang là giá trị mặc định(null hoặc 0) do Constructor của Dog chưa chạy. Điều này cực kỳ dễ dẫn đến lỗi NullReferenceException hoặc sai lệch logic hệ thống rất khó kiểm soát

3.4

- Có, hoàn toàn hợp lệ. Trong C#, phương thức ghi đè ở lớp con (override) có quyền ném ra (throw) bất kỳ ngoại lệ nào mà nó muốn, không bị ràng buộc bởi lớp cha.
- Cách đảm bảo chương trình không bị "crash" khi duyệt một danh sách List<Animal>: Để chương trình chạy xuyên suốt không bị sập giữa chừng khi duyệt danh sách, chúng ta phải bọc lời gọi hàm trong khối try-catch ngay bên trong vòng lặp